SET 9

PART A — Short questions and answers

Q1. Illustrate a one-to-many relationship using CUSTOMER and ORDERS.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q2. Describe the use of the DEFAULT constraint for Order_Status.

Answer: DEFAULT supplies a value automatically when an INSERT does not specify one.

Q3. Describe a deletion anomaly that occurs when the only order of a product is deleted from a combined table.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q4. Describe the use of the LIKE operator with the wildcard % to search customer names.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q5. Interpret the query: SELECT City, COUNT(*) FROM CUSTOMER GROUP BY City HAVING COUNT(*) > 2;

Answer: COUNT(*) counts all rows, including rows containing NULL values in some columns.

Q6. Explain how the NO_DATA_FOUND exception is used in a procedure that accepts a CustomerID.

Answer: PL/SQL adds procedural programming to SQL. A procedure is a reusable named block that can take parameters.

Q7. Summarize the main types of NoSQL databases.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q8. Describe the use of the updateOne() method with the $set operator.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q9. Describe the output of the MongoDB query: db.orders.find({OrderStatus: {$ne: "Delivered"}})

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Explain how a relationship property such as [:BOUGHT {qty: 2}] is useful.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

PART B — System solution

Scenario: An online retail store maintains a E-Commerce Order Management System to store customer details,

1(a) ER design:

• **CUSTOMER**(CustomerID, CustomerName, Email, City); primary key: **CustomerID**.

• **PRODUCT**(ProductID, ProductName, Category, Price); primary key: **ProductID**.

• **ORDERS**(OrderID, CustomerID, ProductID, OrderDate, Quantity, OrderStatus); primary key: **OrderID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE CUSTOMER (
  CustomerID NUMBER PRIMARY KEY,
  CustomerName VARCHAR2(100),
  Email VARCHAR2(100),
  City VARCHAR2(100)
);
CREATE TABLE PRODUCT (
  ProductID NUMBER PRIMARY KEY,
  ProductName VARCHAR2(100),
```

```
  Category VARCHAR2(100),
  Price NUMBER
);
CREATE TABLE ORDERS (
  OrderID NUMBER PRIMARY KEY,
  CustomerID NUMBER,
  ProductID NUMBER,
  OrderDate DATE,
  Quantity NUMBER,
  OrderStatus VARCHAR2(100)
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM ORDERS x
JOIN CUSTOMER a ON x.CustomerID = a.CustomerID
JOIN PRODUCT b ON x.ProductID = b.ProductID;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. MongoDB: Create the collections and insert documents using the fields listed in the set. Query with find() and the requested filter.

```
db.collection.insertMany([
  { _id: 1, name: 'Sample A', category: 'Example', price: 500 },
  { _id: 2, name: 'Sample B', category: 'Example', price: 300 }
]);
db.collection.find({ author: 'Specified Author' });
db.collection.find({ price: { $gt: 400 } });
```